

Fuzz Testing

SOFT 467 / CSCE 467/867

First, some more review...

Kinds of Testing Techniques

- Manual vs Automated Tests
- Functional vs Non-functional

- Unit Tests

single piece of code (think method)

- Integration Tests

many units, together

- Regression Tests

after anything changes

- Acceptance Tests

Does functionality meet business requirements?

Unit Tests

- Tests individual “units of source code” -- often, method(s)
- Test cases “look like other code”
- Utilizes assertions to raise exceptions on test failures

- Pros: usually small and fast, easy to isolate errors
- Cons: often lots of code per line being tested, hard to test UIs, might require changes to code/architecture to enable testing

Unit Tests in Python

```
import unittest
```

```
from my_pkg import sum
```

```
class TestSum(unittest.TestCase):
```

```
    def test_list_int(self):
```

```
        data = [1, 2, 3]
```

```
        result = sum(data)
```

```
        self.assertEqual(result, 6)
```

```
if __name__ == '__main__':
```

```
    unittest.main()
```

Unit Tests in Python

...

```
def test_single_int(self):  
    result = sum(5)  
    self.assertEqual(result, 5)
```

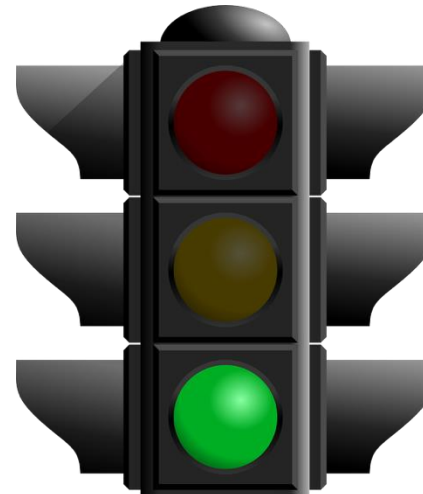
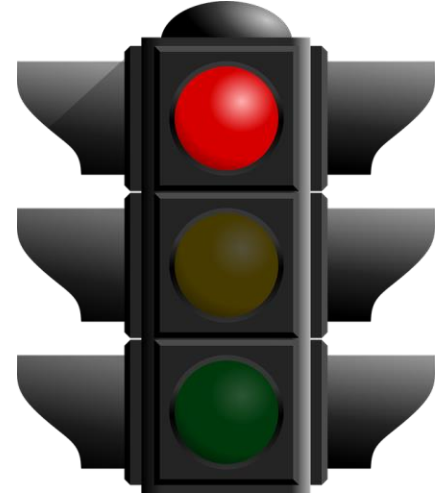
Unit Tests in Python

...

```
def test_list_empty (self):  
    with self.assertRaises(AssertionError):  
        result = sum([])
```

Test-Driven Development (TDD)

- Encode requirements as a unit test
 - Make sure test fails
- Design and implement the API(s) needed
- Test passes



Regression Tests

Haven't I fixed this before ... ?

- Run after any code change
- For any new bug, should write test to first reproduce that bug
- Then fix the bug (test passes!)
- That test now serves as a regression test

Integration Tests



Integration Tests



Mocking

- Focus on a single unit of code
- Don't care what other parts do, as long as they give us something we want/expect and can then use in our test!

Mocking in Python

```
from unittest.mock import MagicMock
```

```
class C: pass
```

```
obj = C()
```

```
obj.foo()
```

```
obj.foo = MagicMock()
```

```
obj.foo()
```

```
obj.foo.return_value = 3
```

```
obj.foo()
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

AttributeError: 'C' object has no attribute 'foo'

<MagicMock name='mock()' id='4473497008'>

3